



Hackatweet — Plan d'équipe

Matthieu & Mohammed · hackathon La Capsule · Les cases se cochent et restent cochées sur ce navigateur — chacun ouvre le lien et suit sa colonne.

PHASE 0 — ENSEMBLE ~30–45 min, avant de se séparer

- ☐ Repo GitHub `hackatweet` créé, les deux devs collaborateurs, clonés chacun
- ☐ Frontend : `create-next-app` (pages router, comme `morningnews`) — Backend : `express-generator` + `mongoose`, `dotenv`, `cors`, `bcrypt`, `uid2`
- ☐ Base MongoDB Atlas créée, `CONNECTION_STRING` dans un `.env` chez chacun — et `.env` dans le `.gitignore` **avant** le premier push
- ☐ Les 4 décisions ci-dessous tranchées et notées
- ☐ Contrat API relu et validé à deux (c'est votre contrat de travail : personne ne le change sans prévenir l'autre)
- ☐ Rôles choisis : qui prend la piste bleue, qui prend la rose

4 DÉCISIONS À TRANCHER ENSEMBLE

- 1. Schéma User**

La modale signup vous donne les champs saisis — qu'ajoute le backend ? (indice : sécurité ×2)
- 2. Auteur d'un tweet**

Copier le username dans le tweet, ou référence + `populate` comme dans `morningnews` ? Que se passe-t-il si un user change ?
- 3. Type de likes**

Le cœur devient rouge seulement si *l'utilisateur connecté* a liké → un simple compteur suffit-il ?
- 4. Les hashtags**

Collection dédiée en BDD ou calculés à la volée depuis les tweets ? Testez votre choix contre : « un tweet avec `#hello` est supprimé ».

PISTE A — BACKEND

0/9

Tout se teste dans Thunder Client / Postman au fur et à mesure — pas besoin d'attendre le front.

- ☐ Connexion MongoDB + modèles `User` et `Tweet`
- ☐ `POST /users/signup` : `checkBody`, refus si `username` déjà pris, hash `bcrypt`, token `uid2`
- ☐ `POST /users/signin` : compare `bcrypt`, renvoie le token

⚡ SYNC 1 — L'AUTH PASSE DE BOUT EN BOUT

- ☐ `POST /tweets` : retrouver l'auteur via son token, refuser si vide ou > 280
- ☐ `GET /tweets` : auteur peuplé, tri du plus récent au plus ancien

⚡ SYNC 2 — LE FIL AFFICHE DE VRAIS TWEETS

- ☐ `PUT /tweets/like` : toggle — ajoute le user s'il n'a pas liké, le retire sinon
- ☐ `DELETE` : vérifier que le token correspond bien à l'auteur du tweet
- ☐ Route hashtag : tweets contenant `#xxx`, insensible à la casse

⚡ SYNC 3 — LIKE, DELETE, TRENDS, HASHTAG BRANCHÉS

- ☐ Bonus TDD : tests sur la vérification des champs (le brief le demande)

PISTE B — FRONTEND

0/10

Avant chaque sync, on peut avancer avec des données en dur, puis on branche les fetch.

- ☐ Store Redux : reducer `user` (token, username, firstname) + `redux-persist` si le temps
- ☐ Page Login : layout 2 colonnes, logo retourné, image de fond, police Mont-Heavy
- ☐ Modales `SignUp` / `SignIn` : inputs contrôlés, ouverture/fermeture

⚡ SYNC 1 — FETCH SIGNUP/SIGIN, TOKEN DANS REDUX, REDIRECTION HOME

- ☐ Layout Home 3 colonnes : logo-lien, infos user, bouton logout
- ☐ Zone « What's up? » : compteur `x/280`, saisie bloquée à 280
- ☐ Composant `Tweet` : auteur, date relative, hashtags en bleu dans le texte

⚡ SYNC 2 — FETCH GET/POST TWEETS, LE FIL EST RÉEL

☐ Cœur rouge si liké par moi, poubelle seulement sur mes tweets

☐ Composant Trends : hashtags + nombre de tweets

☐ Route dynamique /hashtag/[query] + barre de recherche qui met à jour l'URL

⚡ SYNC 3 — PARCOURS COMPLET À DEUX, ÉCRAN PAR ÉCRAN FACE À LA VIDÉO

☐ « No tweets found with #xxx » si aucun résultat

CONTRAT API réponses au format La Capsule : toujours un champ result

MÉTHODE	ROUTE	BODY / PARAMS	RÉPONSE (SUCCÈS)
POST	/users/signup	firstname, username, password	{ result, token }
POST	/users/signin	username, password	{ result, token }
GET	/tweets	—	{ result, token }
POST	/tweets	token, content	{ result }
PUT	/tweets/like	token, tweetId	{ result }
DELETE	/tweets/:tweetId	token	{ result }
GET	/tweets/hashtag/:query	—	{ result, token }
GET	/tweets/trends	—	optionnelle — trends peuvent être obtenus depuis GET /